

SAMA7G54-EK Evaluation Kit

Developer Setup & Getting Started Guide

Version 1.0 | April 2026

1. Overview

This guide provides step-by-step instructions for setting up the SAMA7G54-EK Evaluation Kit for bare-metal and MPLAB Harmony v3 application development. It covers hardware setup, bootloader (at91bootstrap) configuration, flashing, and running your first LED application.

2. Hardware Overview

2.1 Board Connectors

Connector	Purpose	Notes
J1	5V DC Power Input	2.1mm coaxial connector
J4	SD Card Slot	Must be empty for QSPI boot
J7	USB-A (micro-B)	Power + SAM-BA programming
J24	USB J-Link OB + CDC	Debugger + Serial console (COM port)
J27	JTAG 20-pin	External JTAG debugger

2.2 Buttons

Button	Function
SW1	User Button (PA12) - free use in application
SW2	nReset - resets the processor
SW3	nStart - wakes up processor from shutdown
SW4	Disable Boot - disables boot from eMMC and NOR Flash (SAM-BA mode)

2.3 LEDs

LED	Color	Pin	Logic
D7	Blue	PD20	Active HIGH
D7	Green	PA13	Active HIGH
D7	Red	PA6	Active HIGH

2.4 Jumper Configuration

Jumper	Default	Function
J22	Open	Open = MPU boots from onboard memory. Closed = disables boot
J25	Open	Open = enables serial comms (J-Link CDC)
J26	Open	Open = enables J-Link OB debug probe
J3	Open	Open = Ethernet 10/100. Closed = PDMC microphones

WARNING: J22 must be OPEN for normal boot from QSPI. Keep J25 and J26 OPEN for debugging and console output.

3. Software Prerequisites

Install the following tools before starting:

Tool	Version Used	Download
MPLAB X IDE	v6.30	https://www.microchip.com/mplab/mplab-x-ide
XC32 Compiler	v5.10	https://www.microchip.com/mplab/compilers
SAM-BA ISP	v3.9.2	https://www.microchip.com/en-us/development-tool/SAM-BA-In-system-Programmer
PuTTY	Any	https://www.putty.org/
Git	Any	https://git-scm.com/

4. Cable Setup

You need TWO USB cables connected simultaneously:

Cable	From Board	Purpose
Cable 1	J7 (micro-USB)	Power the board + SAM-BA programming port
Cable 2	J24 (micro-USB)	J-Link debugger + Serial console (COM port)

Note: When J24 is connected, Windows will show a JLink CDC UART Port (e.g. COM4) in Device Manager. Use this port for PuTTY console.

5. Console Serial Setup (PuTTY)

Open PuTTY and configure:

Setting	Value
Connection type	Serial
Serial line	COM4 (check Device Manager for exact port)
Speed (Baud rate)	115200
Data bits	8
Stop bits	1
Parity	None
Flow control	None

6. Building at91bootstrap (Bootloader)

6.1 What is at91bootstrap?

at91bootstrap is the second-stage bootloader for the SAMA7G54. It initializes clocks, DRAM, and loads the application binary from QSPI flash into RAM before jumping to it.

6.2 Clone the Repository

```
git clone https://github.com/linux4sam/at91bootstrap.git
```

6.3 Open in MPLAB X IDE

1. In MPLAB X IDE, go to File > Open Project
2. Select the at91bootstrap folder
3. Click Open Project

6.4 Configure Project Properties

Device Settings

4. Right-click at91bootstrap project > Properties
5. Set Device to: SAMA7G54
6. Select DFP version 1.13.310 and click Apply

Makefile Settings

7. In Categories, click Makefile
8. **Build command:** make CROSS_COMPILE="C:/Program Files/Microchip/xc32/v5.10/bin/bin/pic32c-"
9. **Clean command:** make mplabclean CROSS_COMPILE="C:/Program Files/Microchip/xc32/v5.10/bin/bin/pic32c-"

Note: Replace v5.10 with your actual XC32 version. Check via Tools > Options > Embedded > Build Tools.

Kconfig Settings

10. In Categories, click Kconfig
11. Click Load and navigate to configs/ folder
12. Select the appropriate defconfig:

Config File	Use Case
sama7g5ek_bkptnone_defconfig	JTAG debugging only (configure and stop)
sama7g5ekdf_qspi_uboot_defconfig	Boot application from QSPI flash (production)

Config File	Use Case
sama7g5eksd_uboot_defconfig	Boot application from SD card

6.5 Build

13. Right-click at91bootstrap > Clean and Build
14. **Output binary location:** at91bootstrap/build/binaries/boot.bin

7. Flashing with SAM-BA

7.1 Enter SAM-BA Boot Mode

15. Press and hold SW4 (Disable Boot)
16. While holding SW4, press and release SW2 (Reset)
17. Release SW4
18. Check PuTTY - you should see: RomBOOT (confirms SAM-BA mode is active)

7.2 Flash at91bootstrap to QSPI

Open CMD and navigate to the binaries folder:

```
cd C:\Users\<user>\Downloads\at91bootstrap-master\build\binaries
```

Erase and flash:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a qspiflash -c "erase:0x0:0x40000" -c "writeboot:boot.bin"
```

Verify the flash:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a qspiflash -c "verifyboot:boot.bin"
```

Note: Replace COM4 with your actual COM port number from Device Manager.

7.3 Flash Application Binary (harmony.bin) to QSPI

After building your Harmony v3 application, flash harmony.bin at offset 0x40000:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a qspiflash -c "erase:0x40000:0xC0000" -c "write:harmony.bin:0x40000"
```

8. Boot Configuration Packet (BCP)

8.1 Configure Boot from QSPI (Emulation Mode - Development)

WARNING: Emulation mode must be reconfigured after every full power cycle. Use OTP for permanent configuration.

Enter SAM-BA mode first, then run these commands in order:

19. Enable emulation mode:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a bootconfig -c writecfg:bscr:EMULATION_ENABLED -c resetemul -c refreshcfg:emul -c writecfg:bcp-emul:QSPI0_IOSET1 -c readcfg:bcp-emul
```

20. Expected output:

```
BCP-EMUL=FLEXCOM0_USART_IOSET1,QSPI0_IOSET1
```

8.2 Configure Boot from QSPI Permanently (OTP)

WARNING: OTP is ONE-TIME PROGRAMMABLE and CANNOT be undone. Only use for production!

21. Disable emulation:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a bootconfig -c  
writecfg:bscr:EMULATION_DISABLED
```

22. Refresh OTP:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a bootconfig -c refreshcfg:otp
```

23. Write BCP to OTP:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a bootconfig -c writecfg:bcp-  
otp:QSPI0_IOSET1
```

24. Verify:

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a bootconfig -c readcfg:bcp-otp
```


9. Normal Boot Sequence

After flashing everything:

25. Press SW2 (Reset) only — do NOT hold SW4

26. Watch PuTTY console for at91bootstrap output:

```
MCK: mck domains initialization complete.  
TZC400: Initialization complete.  
PUBL: Training complete.  
UMCTL2: Initialization complete.  
AT91Bootstrap 4.0.13-rc1 (...)  
SF: Got Manufacturer and Device ID: c2853b  
SF: Copy 0xc0000 bytes from 0x40000 to 0x66f00000  
SF: Done to load image
```

27. Application starts running from RAM

10. MPLAB Harmony v3 Application Development

10.1 Clone Example Applications

```
mkdir C:\Users\<user>\Harmony3  
cd C:\Users\<user>\Harmony3  
git clone https://github.com/Microchip-MPLAB-Harmony/csp_apps_sam_a7g5
```

10.2 Open Example Project

28. In MPLAB X IDE: File > Open Project

29. Navigate to:

Harmony3\csp_apps_sam_a7g5\apps\pio\pio_led_on_off_interrupt\firmware\sam_a7g5_ek.X

30. Click Open Project

10.3 Configure Project for Debugging

31. Right-click project > Properties

32. Select latest DFP (1.13.310) and click Apply

33. Set Connected Hardware Tool to: J-Link On-Board (OB)

34. Under Bootstrap category: verify Use Bootstrap is checked with at91bootstrap.elf

35. Go to Tools > Options > Embedded > Generic Settings:

- Debug Reset @ = Main
- Debug Startup = Halt at Main

10.4 Debug and Run

36. Press SW3 (nStart) to wake up the board

37. Click the Debug button (bug icon) in MPLAB X
38. Wait for yellow arrow to appear at main() - this is Halt at Main
39. Click Continue (green play button) or press F5
40. Press SW1 (User Button) - RGB LED should light up!

10.5 Generate Production Binary (harmony.bin)

41. In Project Properties > Building
42. Check Execute this line after build
43. Add this command:

```
${MP_CC_DIR}/xc32-objcopy -O binary ${DISTDIR}/${PROJECTNAME}.${IMAGE_TYPE}.elf  
${DISTDIR}/harmony.bin
```
44. Click OK
45. Right-click project > Clean and Build
46. harmony.bin will be generated in the dist/ folder

11. LED Example Code

11.1 Simple LED Blink (Register Level)

For bare-metal projects using direct register access:

```
#include <sama7g54.h>

#define LED_PIN      (13U)    // PA13 = Green LED
#define LED_MASK     (1UL << LED_PIN)

void delay(volatile unsigned int count) { while(count--); }

int main(void)
{
    // Enable PIOA clock for PIOA (PID=11)
    PMC_REGS->PMC_PCR = PMC_PCR_PID(11) | PMC_PCR_CMD_Msk | PMC_PCR_EN_Msk;

    // Configure PA13 as GPIO output
    PIO_REGS->PIO_GROUP[0].PIO_MSKR = LED_MASK;
    PIO_REGS->PIO_GROUP[0].PIO_CFGR = (0x0 << PIO_CFGR_FUNC_Pos) |
    PIO_CFGR_DIR_Msk;

    while(1)
    {
        PIO_REGS->PIO_GROUP[0].PIO_MSKR = LED_MASK;
        PIO_REGS->PIO_GROUP[0].PIO_ODSR = LED_MASK; // LED ON
        delay(500000);
        PIO_REGS->PIO_GROUP[0].PIO_ODSR = 0;          // LED OFF
        delay(500000);
    }
    return 0;
}
```

11.2 Button-Controlled LED (Harmony v3)

For Harmony v3 projects using MCC-generated code:

```
#include "definitions.h"

#define SWITCH_PRESSED_STATE 0
#define LED_On()              LED_BLUE_Set()
#define LED_Off()             LED_BLUE_Clear()

void delay(volatile uint32_t count) { while(count--); }
```

```
void controlled(PIO_PIN pin, uintptr_t context)
{
    if(SWITCH_Get() == SWITCH_PRESSED_STATE)
    {
        LED_On();
        delay(2000000); // Stay ON for ~2 seconds
        LED_Off();
    }
    else { LED_Off(); }
}

int main(void)
{
    SYS_Initialize(NULL);
    PIO_PinInterruptCallbackRegister(SWITCH_PIN, controlled, (uintptr_t)NULL);
    PIO_PinInterruptEnable(SWITCH_PIN);
    while(true) {}
    return EXIT_FAILURE;
}
```

12. Troubleshooting

Problem	Cause	Solution
RomBOOT repeating in PuTTY	Boot Configuration Packet not set	Run bootconfig commands to set QSPI boot (Section 8)
SAM-BA: Access is denied on COM4	Port in use by another program	Close PuTTY or MPLAB X before running SAM-BA
SAM-BA: Could not initialize applet (status:1)	Board not in SAM-BA mode	Redo SW4+SW2 sequence and try again
SAM-BA: Incomplete QSPI configuration	Missing instance/ioset/freq parameter	Use -b sama7g54-ek flag instead of manual params
BUILD FAILED: pic32c-gcc not found	Wrong compiler path in Makefile	Check XC32 version and path (bin/bin/ for v5.10)
Nothing in PuTTY console	Wrong COM port or J22 jumper issue	Check Device Manager for correct port, ensure J22 is open
LED not lighting	Wrong pin or logic level	Check LED pin table (Section 2.3) and active level
Debug halts but LED doesn't blink	Forgot to click Continue	Press F5 or click green Continue button

13. Quick Reference Commands

Flash at91bootstrap

```
cd <at91bootstrap>/build/binaries
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a qspiflash -c "erase:0x0:0x40000" -c
"writeboot:boot.bin"
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a qspiflash -c "verifyboot:boot.bin"
```

Flash Application

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a qspiflash -c "erase:0x40000:0xC0000" -c
"write:harmony.bin:0x40000"
```

Set Boot Config (run after every power cycle in dev mode)

```
sam-ba.exe -p serial:COM4 -b sama7g54-ek -a bootconfig -c
writecfg:bscr:EMULATION_ENABLED -c resetemul -c refreshcfg:emul -c writecfg:bcpr-
emul:QSPI0_IOSET1
```

14. References

- SAMA7G54-EK Features: <https://developerhelp.microchip.com/xwiki/bin/view/software-tools/mcu-dev-boards/32-bit-kits/sama7g54-ek/features/>
- at91bootstrap Build Guide: <https://developerhelp.microchip.com/xwiki/bin/view/software-tools/32-bit-harmony-v3/sama7g54-ek-h3-build-at91bootstrap/>
- AT91Bootstrap v4.x:
<https://developerhelp.microchip.com/xwiki/bin/view/applications/linux4sam/components/at91bootstrap/>
- Write NOR Flash with SAM-BA:
<https://developerhelp.microchip.com/xwiki/bin/view/software-tools/32-bit-harmony-v3/sama7g54-ek-h3-write-nor/>
- CSP LED Example: <https://developerhelp.microchip.com/xwiki/bin/view/software-tools/32-bit-harmony-v3/sama7g54-ek-h3-csp-app-pio-led-on-off-interrupt/>
- at91bootstrap GitHub: <https://github.com/linux4sam/at91bootstrap>
- CSP Apps GitHub: https://github.com/Microchip-MPLAB-Harmony/csp_apps_sam_a7g5